Programación 1 — Unidad 9: Manejo de archivos

Actividad 1: Introducción al manejo de archivos

Índice

9. Manejo de archivos

– 9.1. Por qué un programa necesita archivos

– 9.2. Memoria vs. almacenamiento permanente

– 9.3. Tipos de archivos: texto plano y CSV



9. Manejo de archivos

Hasta ahora todo lo que hicieron tus programas desapareció al cerrarlos. Cargabas una lista, calculabas un promedio, mostrabas un resultado… y al volver a ejecutar, arrancabas de cero. En esta unidad eso cambia: vas a aprender a **guardar información en el disco** para que sobreviva entre una ejecución y otra.

Esta primera actividad es conceptual. Antes de escribir código conviene entender **qué problema resuelven los archivos** y **qué tipos de archivo** vas a usar. En las actividades siguientes vas a ponerlo en práctica.

9.1. Por qué un programa necesita archivos

Mientras un programa se ejecuta, sus datos viven en la memoria RAM y **se pierden** al cerrarlo. Si querés que la información sobreviva entre una ejecución y otra —una lista de contactos, una configuración, un registro de ventas— hay que guardarla en **archivos** en el disco. A esa capacidad de que los datos perduren se la llama **persistencia**.

Pensá en programas que usás todos los días:

- Un **editor de texto** guarda lo que escribís en un archivo; si no, perderías el documento al cerrarlo.
- Un **videojuego** guarda tu progreso: al volver, seguís donde estabas.
- Una **agenda de contactos** conserva los números que cargaste la semana pasada.

En los tres casos pasa lo mismo: el programa **escribe** los datos en el disco antes de cerrarse y los **lee** de vuelta cuando arranca. Eso es exactamente lo que vas a aprender a hacer.

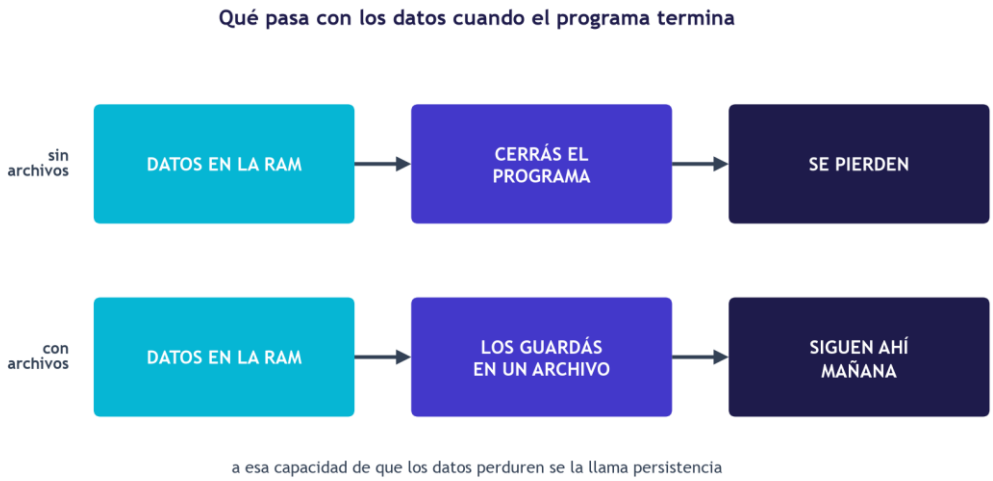


Diagrama 9.1: sin archivos los datos se pierden al cerrar; con archivos sobreviven.

Analogía

La memoria RAM es como el **pizarrón** de un aula: sirve para trabajar mientras dura la clase, pero al final del día se borra. El disco es la **carpeta** donde anotás lo importante para poder consultarlo mañana. Guardar un archivo es pasar del pizarrón a la carpeta.

9.2. Memoria vs. almacenamiento permanente

Una computadora guarda datos en dos lugares muy distintos, y confundirlos es una fuente clásica de errores. Esta es la diferencia:

	Memoria RAM	Almacenamiento (disco)
Cuándo vive el dato	Solo mientras el programa se ejecuta	Hasta que alguien lo borre
Al apagar la máquina	Se pierde todo	Se conserva
Velocidad	Muy rápida	Más lenta
Capacidad típica	Chica (GB)	Grande (cientos de GB o TB)
Ejemplo en tu código	Una lista, un diccionario, una variable	Un archivo `.txt`, `.csv` o `.json`

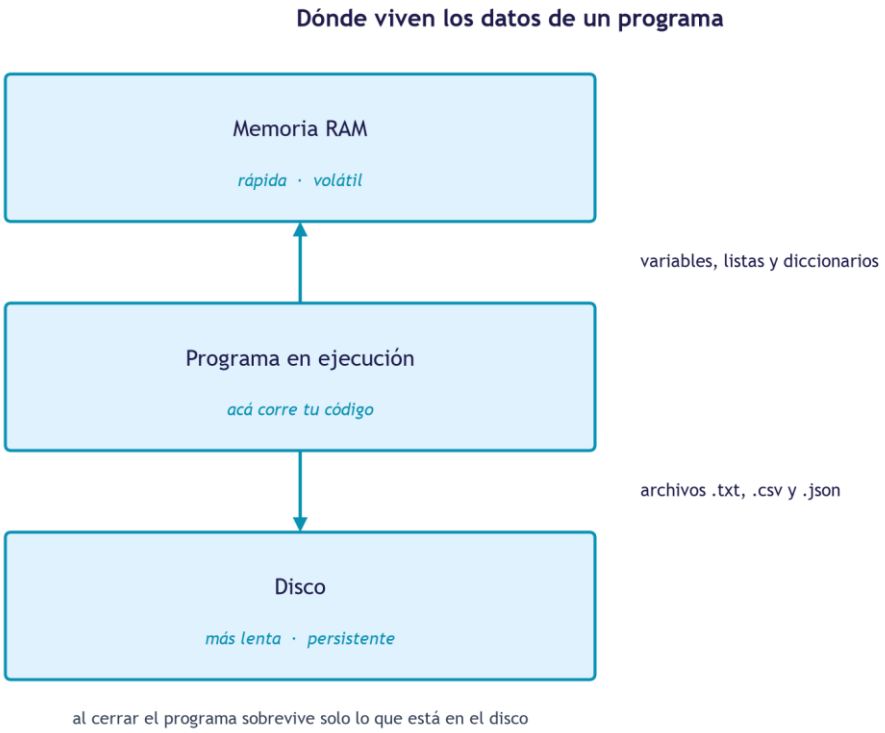


Diagrama 9.2: el programa trabaja en la RAM y guarda en el disco lo que necesita conservar.

Cuando escribís `notas = [7, 8, 10]`, esa lista está en la RAM: es rápida de recorrer, pero **volátil**. Cuando la guardás en un archivo, pasa al disco: tarda un poco más, pero es **persistente**.

Importante

Guardar un dato en una **variable** no es guardarlo en un archivo. La variable vive en la RAM y desaparece cuando el programa termina. Para que un dato persista hay que **escribirlo explícitamente en un archivo**; Python no lo hace solo.

9.3. Tipos de archivos: texto plano y CSV

Existen muchísimos formatos de archivo, pero en esta unidad vamos a trabajar con dos, y los dos son **texto plano**: se pueden abrir con cualquier editor y leer con los ojos, sin ningún programa especial.

- **Texto plano (`.txt`)**: contienen caracteres legibles. Ideales para notas, listas o mensajes simples. No tienen estructura interna: son líneas de texto, una atrás de la otra.
- **CSV (`.csv`)**: del inglés *Comma Separated Values*. Guardan datos en forma de tabla: cada línea es una fila y los valores se separan con comas. Se abren con cualquier planilla de cálculo, como Excel.

Así se ve un `.txt` con una lista de tareas:

```
Comprar pan
Estudiar Programación 1
Llamar a Ana
```

Y así el mismo tipo de información, pero en `.csv`, con estructura de tabla:

```
nombre,edad,pais
Ana,30,Argentina
Juan,25,Argentina
Lucia,41,Uruguay
```

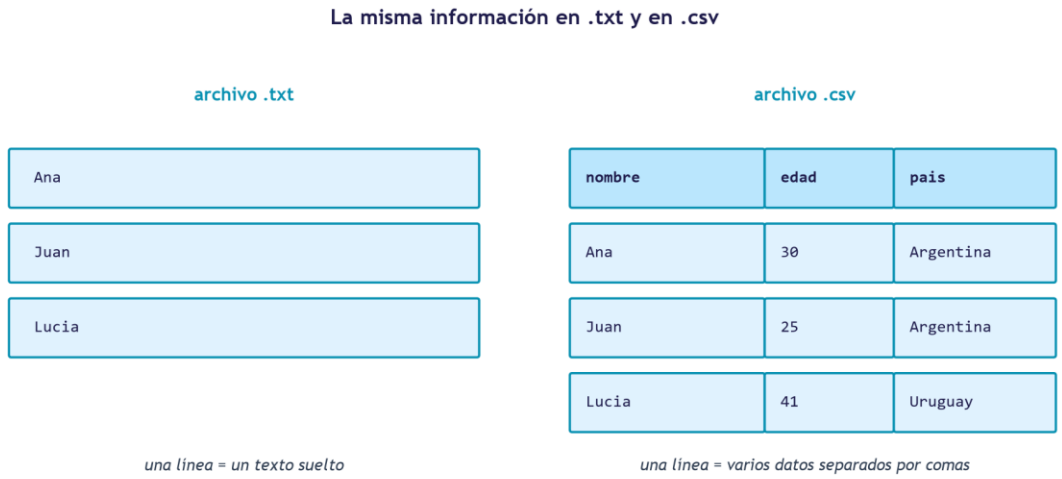


Diagrama 9.3: el .txt guarda líneas sueltas; el .csv guarda filas con columnas.

Fijate la diferencia clave: en el `.txt` cada línea es **un texto suelto**; en el `.csv` cada línea son **varios datos separados por comas**, y la primera línea dice qué significa cada uno. Esa primera línea se llama **cabecera** o *header*.

Qué hace el código:

- El `.txt` sirve cuando la información no tiene columnas: notas, mensajes, un registro de eventos.
- El `.csv` sirve cuando la información **es una tabla**: cada fila un registro, cada columna un campo.
- Los dos son texto plano, así que se leen y escriben con las mismas herramientas básicas de Python.
- Un `.csv` lo podés abrir con Excel o Google Sheets y lo vas a ver como una planilla, con cada valor en su celda.

Nota

Un archivo `.csv` **no** es un formato de Excel. Es texto plano, y por eso cualquier lenguaje y cualquier programa lo puede leer. Un `.xlsx` en cambio es un formato binario propio de Excel, mucho más pesado y más difícil de procesar desde código. Esa portabilidad es la razón por la que el CSV se usa tanto para intercambiar datos entre sistemas.

Para reflexionar y autoevaluarte

— ¿Qué información de un programa que hayas escrito te hubiera servido conservar entre una ejecución y otra?

— Si tuvieras que guardar las notas de todos los alumnos de un curso, ¿elegirías `.txt` o `.csv`? ¿Por qué?

Antes de seguir, verificá que podés:

— Explicar con tus palabras qué es la persistencia.

— Distinguir qué datos viven en la memoria RAM y cuáles en el disco.

— Reconocer cuándo conviene un archivo de texto plano y cuándo un CSV.

En la **Actividad 2** vas a pasar de la teoría a la práctica: abrir un archivo desde Python, escribir en él, leerlo y cerrarlo de forma segura.